The Categorical Structure of Layered Recovery: Stratified Endofunctors on Quantale-Enriched Lattices

The Prologos Project - From Zachary Larson

2026-03-13

Contents

1 Abstract **2**

2 1. The Three Systems **3**
2.1 1.1 System I: The Logic Engine (`rel`) 3
2.2 1.2 System II: The Session Runtime (`proc`) 4
2.3 1.3 System III: The Type Checker (Stratified Quiescence) . . 6

3 2. The Common Categorical Structure **8**
3.1 2.1 Definition: Stratified Recovery System 8
3.2 2.2 The Enrichment: Quantale Structure 8
3.3 2.3 The Decomposition: Inter-Stratum Galois Connections . . 9
3.4 2.4 Theorem: Stratification Preserves Fixpoint Equivalence . 10
3.5 2.5 The Common Structure: Diagram 10

4 3. The Quantale Morphism as Bridge **11**
4.1 3.1 Why Quantales, Not Just Lattices? 11
4.2 3.2 The Bridge Pattern in Implementation 12
4.3 3.3 Composition of Galois Connections 13

5 4. Formal Statements **13**
5.1 4.1 Definition: Stratifiable Operation 13
5.2 4.2 Theorem: Generalized Stratification 13
5.3 4.3 Theorem: Quantale Morphism Preservation 14
5.4 4.4 Corollary: Confluence Under Non-Overlapping Resolution 14

6 5. Relationship to Approximation Fixpoint Theory 15
6.1 5.1 The Connection 15
6.2 5.2 Layered Recovery as a Special Case of AFT 15
6.3 5.3 What AFT Offers for Future Extensions 16

7 6. Variants and Approaches for Non-Monotonic Domains 16
7.1 6.1 Classification of Non-Monotone Operations 16
7.1.1 Class A: Stratifiable (barrier-compatible) 16
7.1.2 Class B: Locally Stratifiable (barrier-compatible with context restriction) 17
7.1.3 Class C: Anti-Monotone (requires bilattice or ATMS) . 17
7.1.4 Class D: Chaotic (no fixpoint guarantee) 17
7.2 6.2 The Adaptation Recipe 18
7.3 6.3 Specific Future Applications in Prologos 18

8 7. Implications and Conclusions 19
8.1 7.1 What the Categorical Structure Tells Us 19
8.2 7.2 What This Means for Implementation 19
8.3 7.3 What This Means for Theorists 20
8.4 7.4 Open Questions 20

9 8. References 20
9.1 8.1 Categorical and Algebraic Foundations 20
9.2 8.2 Effect Systems and Graded Monads 21
9.3 8.3 Fixpoint Theory and Stratification 21
9.4 8.4 CALM Theorem and Distributed Monotonicity 22
9.5 8.5 Propagator Networks 22
9.6 8.6 Session Types 22
9.7 8.7 Datalog and Non-Monotonic Reasoning 22
9.8 8.8 Prologos Internal Documents 23

1 Abstract

We identify the precise categorical structure underlying the Layered Recovery Principle as implemented in three subsystems of the Prologos language: the logic engine (stratified negation), the session runtime (effect ordering), and the type checker (reactive constraint resolution). Each system faces the same fundamental problem: recovering non-monotone behavior on a monotone propagator substrate.

We show that all three systems instantiate a common categorical pattern: a **stratified endofunctor** on a **quantale-enriched lattice**, where the endofunctor decomposes as a chain of monotone functors between lattices connected by Galois connections, with non-monotone operations isolated at precisely one barrier in the chain. The global fixpoint is the least fixpoint of the endofunctor, computed by Kleene iteration over the stratified chain.

We prove that this structure is sufficient to recover any non-monotone operation on a monotone substrate, provided the non-monotone operation satisfies a *stratifiability condition*: it must be expressible as a barrier between two monotone phases. We characterize the class of non-monotone operations that satisfy this condition and identify variants for operations that do not.

2 1. The Three Systems

Three subsystems of Prologos independently discovered and implemented the same architectural pattern. We analyze each in turn, extracting the precise algebraic structure.

2.1 1.1 System I: The Logic Engine (`rel`)

Non-monotone operation: Negation-as-failure (NAF). If P is not derivable at the current stratum, conclude ¬P. This is non-monotone because adding a fact P to the database can cause ¬P to become invalid.

Implementation: Stratified evaluation with three layers.

```
Layer 1 (Propagator Network): Unification constraints propagate through cells.
  Values refine monotonically. The network converges to a fixpoint.

Layer 2 (ATMS): Choice points create assumptions. Adding nogoods (inconsistent
  combinations) shrinks the set of valid worldviews - still monotone (the nogood
  set only grows, the valid worldview set only shrinks).

Layer 3 (Stratification Controller): Evaluate ¬P only after stratum containing
  P has reached fixpoint. This is the non-monotone barrier.
```

Lattice structure: Let U be the unification lattice (substitutions ordered by refinement). Each stratum S operates on U. The join is unification; the bottom is the empty substitution.

The stratified chain:

```
S lfp→ U* neg→ S lfp→ U* neg→ S lfp→ ...
 ↑                                                          ↓
  Kleene iteration
```

Where:

- lfp denotes the least fixpoint of monotone propagation within a stratum
- neg denotes the non-monotone negation evaluation at the barrier
- U* is the fixpoint of stratum k

Algebraic structure:

- The unification lattice U is a quantale: substitutions compose (sequential application) and this composition distributes over joins (most general unifiers). The ordering is refinement: iff is a refinement of .
- Negation evaluation neg : U* → Init(U) is not a quantale morphism (it is anti-monotone on the truth ordering). It maps fixpoint states to initial conditions for the next stratum.
- Each stratum's propagation is an endofunctor F : U → U that is monotone. The fixpoint U* = lfp(F).

Sources: Logic Engine Design (`2026-02-24_LOGIC_ENGINE_DESIGN.org` §5), Effectful Computation on Propagators (`EFFECTFUL_COMPUTATION_ON_PROPAGATORS.org` §2).

2.2 1.2 System II: The Session Runtime (`proc`)

Non-monotone operation: IO effect execution. Writing to a file, sending a message, opening a connection — these are irreversible observations that cannot be undone, retried, or merged.

Implementation: Five-layer architecture.

```
Layer 1 (Session Advancement): Session cells advance monotonically through
  protocol states. Each advancement is a Lamport clock tick.

Layer 2 (Data-Flow Analysis): Analyze variable bindings to derive cross-channel
  ordering edges. Monotone edge accumulation.
```

```
Layer 3 (Transitive Closure): Compute the complete partial order over effects
  by propagating ordering edges to closure. Monotone fixpoint.

Layer 4 (ATMS Branching): Maintain per-branch ordering hypotheses. Adding
  nogoods only shrinks valid worldviews - monotone.

Layer 5 (Effect Handler): Execute effects in a valid linearization of the
  resolved partial order. This is the non-monotone barrier.
```

Lattice structure: Two lattices connected by a Galois connection.

Let S be the *session lattice*: session types ordered by advancement (bot → concrete → advanced states → end). The join is the most-advanced common state.

Let E be the *effect position lattice*: sets of ordering constraints (i < j) over effect indices, ordered by inclusion. The join is set union. Transitive closure is a closure operator on E (monotone, extensive, idempotent).

The Galois connection (,):

```
 : S → E     (extract causal position from session state)
 : E → S     (reconstruct remaining protocol from effect position)

(s)  e    s  (e)    (adjunction property)
```

Both and are monotone. The key insight: preserves sequential composition.

Algebraic structure:

- Both S and E are **effect quantales** (Katsumata 2014, Gordon 2021):
  - Lattice structure (partial order with joins)
  - Sequential composition () distributes over joins
  - Ordering respects composition: i i' j j' i j i' j'
- The Galois connection (,) is a **quantale morphism**: it preserves both the lattice ordering and sequential composition.
- Effect execution (Layer 5) is non-monotone: it selects a linearization of the partial order and performs irreversible IO.

The stratified chain:

```
S → E tc→ E* exec→ World
↑    Galois     monotone     non-monotone
   connection   closure       barrier

(session advancement propagators - monotone)
```

Where tc denotes transitive closure (a monotone closure operator).

Sources: Session Types as Effect Ordering (`2026-03-06_SESSION_TYPES_AS_EFFECT_ORDERING.org` §4-6), Effectful Computation (`EFFECTFUL_COMPUTATION_ON_PROPAGATORS.org` §3-4).

2.3 1.3 System III: The Type Checker (Stratified Quiescence)

Non-monotone operation: Trait resolution commitment. Selecting a specific instance from the set of candidates is a *choice* — once committed, the choice is irreversible. Instance selection can be anti-monotone: adding a more specific instance can invalidate a previous selection.

Implementation: Stratified quiescence with three strata.

```
Stratum 0 (Type Propagation): Cell writes (meta solutions → type cells),
  unification propagators, cross-domain bridges. Monotone: values only refine
  in the type lattice.

Stratum 1 (Readiness Detection): Constraint-readiness propagators scan cell
  states to derive "constraint C is ready to retry" signals. Monotone: once
  a cell becomes non-bot, readiness is permanent.

Stratum 2 (Resolution Commitment): Consume readiness signals. Execute
  resolution (retry constraint, resolve trait, solve meta). Non-monotone:
  commitment selects one instance, excluding alternatives.
```

Lattice structure: Three lattices forming a chain.

Let T be the *type cell lattice*: metavariable values ordered by refinement (bot → partial types → ground types). The merge is unification.

Let R be the *readiness lattice*: sets of ready-constraint descriptors, ordered by inclusion. The join is set union. Readiness is monotone in T: if a cell is non-bot, the constraint is ready; more refined cells are still non-bot.

Let A be the *action lattice*: multisets of resolution actions (trait resolution, constraint retry, hasmethod resolution), ordered by inclusion. Each action is a data descriptor (free monad pattern) — not an executed effect.

The functors between strata:

```
 : T → R     (extract readiness from type cell states)
 : R → A     (map readiness signals to resolution actions)
 : A → T     (commit resolutions → new type cell writes)
```

- is monotone: more refined type cells produce (weakly) more readiness signals.
- is monotone: more readiness signals produce more action descriptors.
- is **not monotone** in the general case: committing a resolution can invalidate other pending resolutions (e.g., solving meta ?X = Nat might make a trait constraint solvable but also make another constraint unsatisfiable). However, is monotone *per action* — each individual resolution writes a more refined value to a type cell.

The stratified chain (one iteration):

```
T lfp(S0)→ T* → R lfp(S1)→ R* → A commit→ T'
↑                                                    ↓
 Kleene iteration (fuel=100)
```

Algebraic structure:

- T is a quantale: type substitutions compose and composition distributes over the unification join.
- R is a join-semilattice (readiness sets under union). It lacks the sequential composition that would make it a full quantale.
- A is a free monoid (action sequences). The ordering is prefix/subset.
- : T* → R is a Galois connection's upper adjoint (abstraction). The lower adjoint : R → T does not exist in the traditional sense because commitment is non-monotone. Instead, : A → T is a *partial function* — it succeeds only when the action is still valid (re-check guard).

Confluence conditions (proved in Track 2 Phase 9):

- S0: Confluent by construction (lattice merge properties).
- S1: Confluent by construction (readiness is monotone in cell state).

- S2: Confluent under the *non-overlapping instance invariant*: each constraint has at most one valid resolution. Prologos enforces this — monomorphic instances are looked up by key, parametric instances use most-specific-wins with same-specificity ties rejected as ambiguous (HKT-7).

Sources: Track 2 Design (`2026-03-13_TRACK2_REACTIVE_RESOLUTION_DESIGN.md` §3.4), Track 2 Phase 9 (`test-resolution-confluence-01.rkt`).

3 2. The Common Categorical Structure

3.1 2.1 Definition: Stratified Recovery System

A **stratified recovery system** is a tuple (L, F_m, f_{nm}, n) where:

1. **L** is a complete lattice (the *substrate*).
2. $\mathbf{F_m : L \to L}$ is a monotone endofunctor (the *propagation phase*). By Knaster-Tarski, $\mathrm{F_m}$ has a least fixpoint $\mathrm{lfp(F_m)}$.
3. $\mathbf{f_{nm} : L \to L}$ is a (possibly non-monotone) function (the *barrier operation*).
4. **n {}** is the *stratification depth* (number of strata).

The **stratified fixpoint** is computed by:

```
x =
For k = 0, 1, ..., n-1:
  x* = lfp(F_m | initialized at x)
  x = f_nm(x*)
Result = x_n*
```

This is Kleene iteration on the composition $\mathrm{f_{nm}}$ $\mathrm{lfp(F_m)}$, starting from .

3.2 2.2 The Enrichment: Quantale Structure

In all three Prologos systems, the substrate lattice L carries additional algebraic structure: it is a **quantale**.

A quantale is a complete lattice (L, ,) equipped with an associative binary operation (: L × L → L) called *sequential composition* that distributes over arbitrary joins:

```
a  ( b) =  (a  b)     (left distributivity)
( a)  b =  (a  b)     (right distributivity)
```

The three systems' quantale structures:

System	Lattice L	Join ()	Composition ()
Logic	Substitutions	Most general unifier	Substitution application
Sessions	Session states	Most-advanced state	Protocol continuation
Types	Type cells	Unification	Type substitution composition

3.3 2.3 The Decomposition: Inter-Stratum Galois Connections

The stratified fixpoint computation decomposes the endofunctor (f_{nm} lfp(F_m)) through intermediate lattices connected by Galois connections.

In the general case, a stratified recovery system with k strata has intermediate lattices L, L, . . . , L and Galois connections:

```
(, ) : L  L     for i = 0, ..., k-2

where  is monotone (abstraction)
       is monotone (concretization)
      (x)  y    x  (y)
```

The non-monotone operation f_{nm} factors through the final lattice L:

```
f_nm =   ...    barrier    ...
```

where *barrier* : L → L is the non-monotone operation at the top stratum. All other operations in the chain are monotone.

The three systems instantiate this pattern:

System	Lattice chain	Galois connections	Barrier
Logic	U → U	identity (single lattice, iterated)	neg : U* → Init(U)
Sessions	S → E → E*	(,) : S E (quantale morphism)	exec : E* → World
Types	T → R → A	: T* → R (abstraction)	commit : A → T

3.4 2.4 Theorem: Stratification Preserves Fixpoint Equivalence

Theorem (Fixpoint Equivalence). Let (L, F_m, f_{nm}, n) be a stratified recovery system. If $\mathrm{f_{nm}}$ is *stratifiable* — i.e., the composition $\mathrm{f_{nm}}$ $\mathrm{lfp(F_m)}$ is monotone — then the stratified fixpoint equals the direct fixpoint:

```
lfp(f_nm  lfp(F_m)) = stratified_fixpoint(L, F_m, f_nm, n)
```

Proof sketch: Define $\mathrm{G} = \mathrm{f_{nm}}$ $\mathrm{lfp(F_m)} : \mathrm{L} \to \mathrm{L}$. Since $\mathrm{lfp(F_m)}$ is monotone (as a function of the initialization point, by the parametric fixpoint theorem — Cousot & Cousot 1979) and $\mathrm{f_{nm}}$ is stratifiable (monotone when composed with lfp), G is monotone. By Knaster-Tarski, G has a least fixpoint lfp(G). The stratified computation is exactly Kleene iteration of G starting from :

```
x = ,  x = G(x) = f_nm(lfp(F_m | x))
```

For monotone G on a complete lattice, Kleene iteration converges to lfp(G) (possibly at a transfinite ordinal for general complete lattices; at for algebraic/continuous lattices, which all three systems are).

Corollary (Confluence). If the stratified fixpoint equals the direct fixpoint and S0 is confluent (lattice merge properties), then the overall computation is confluent — the result is independent of the order in which propagators fire within each stratum.

Note: The stratifiability condition is not always satisfied. System III (type checker) satisfies it only under the non-overlapping instance invariant. If overlapping instances existed, $\mathrm{f_{nm}}$ $\mathrm{lfp(F_m)}$ would not be monotone (adding type information could change which instance is selected, leading to a different fixpoint). This is why Prologos rejects overlapping instances — it is a *semantic requirement for confluence*, not merely a design choice.

3.5 2.5 The Common Structure: Diagram

```
STRATIFIED RECOVERY
    SYSTEM
```

```
System I: rel    System II: proc  System III: tc
(Logic Engine)   (Sessions)       (Type Checker)



        QUANTALE-ENRICHED LATTICE  (L, , )

 Monotone endofunctor Fm : L → L
 Fixpoint computation: lfp(Fm) via propagator network



           GALOIS CONNECTION CHAIN

           L  L  ...  L
           (, ) monotone pairs



           NON-MONOTONE BARRIER

           barrier : L → L
           (negation / execution /
            commitment)



           KLEENE ITERATION

           Iterate (barrier  lfp)
           until global fixpoint
```

4 3. The Quantale Morphism as Bridge

4.1 3.1 Why Quantales, Not Just Lattices?

The lattice structure alone suffices for fixpoint computation (Knaster-Tarski). The quantale structure — the sequential composition — is what makes the

Galois connections between systems meaningful.

Without , a Galois connection (,) : L M between lattices preserves only order information. With , a **quantale morphism** preserves both order AND sequential composition:

```
(a  b) = (a) ' (b)     (preserves composition)
( a) = ' (a)        (preserves joins)
```

This is critical for Prologos because the systems don't just compute values — they compute *sequences of operations*. Substitution application, protocol continuation, and type-checking steps all have sequential structure. The quantale morphism guarantees that the sequential structure is preserved across the Galois bridge.

In System II (sessions), this means: the session protocol's sequential structure (send, then receive, then end) maps faithfully to the effect ordering's sequential structure (effect at position 0 before effect at position 1 before effect at position 2). The morphism doesn't just preserve ordering — it preserves the *reason* for the ordering.

4.2 3.2 The Bridge Pattern in Implementation

In Prologos, the Galois connection between domains is implemented as a **bridge propagator** — a propagator that watches cells in one domain and writes to cells in another:

```
bridge-propagator : Cell[L] × Cell[M] → Propagator
  When Cell[L] refines from l to l':
    write (l') to Cell[M]      ( is the abstraction)
  When Cell[M] refines from m to m':
    write (m') to Cell[L]      ( is the concretization)
```

The monotonicity of and guarantees that the bridge propagator is a valid monotone propagator — it can participate in the propagator network's fixpoint computation without violating confluence.

Existing bridges in Prologos:

- Type Session bridge (`session-propagators.rkt`)
- Type Interval bridge (from abstract interpretation work)
- Session Effect Position bridge (proposed in Architecture D)

Each bridge is a quantale morphism instantiated as a pair of propagators.

4.3 3.3 Composition of Galois Connections

Galois connections compose: if (,) : L M and (,) : M N, then (,) : L N is also a Galois connection.

This means the inter-stratum connections compose into a single Galois connection from the substrate lattice to the barrier lattice. For System III (type checker):

```
(  ,   ) : T  A

where  : T* → R (readiness extraction)
       : R* → A (action construction)
       : A → R (action interpretation)
       : R → T (resolution commitment)
```

The *composed* Galois connection maps directly from type cell fixpoints to resolution actions. This is exactly what the stratified quiescence loop computes — but the categorical perspective reveals that the intermediate lattice R (readiness) is not essential; it is a *factorization* of the composed connection that makes the computation more efficient (readiness can be computed incrementally by propagators).

5 4. Formal Statements

We now state the key results precisely.

5.1 4.1 Definition: Stratifiable Operation

A function f : L → L on a complete lattice L is *stratifiable with respect to a monotone endofunctor F : L → L* if the composition f lfp(F) : L → L is monotone, where lfp(F) denotes the parametric least fixpoint of F (which is monotone in the initialization point by Cousot & Cousot 1979).

5.2 4.2 Theorem: Generalized Stratification

Let L be a complete lattice, F : L → L monotone, and f : L → L stratifiable with respect to F. Define the stratified iteration:

```
G = f  lfp(F) : L → L

x =
x = G(x)
```

Then:

1. G is monotone (by definition of stratifiability).
2. lfp(G) exists (by Knaster-Tarski).
3. The Kleene chain x x ... converges to lfp(G).
4. lfp(G) is the unique minimal solution to: x = f(lfp(F | x)).

5.3 4.3 Theorem: Quantale Morphism Preservation

Let (Q, ,) and (Q, ,) be quantales and (,) : Q Q a Galois connection that is also a quantale morphism (preserves and). If F : Q → Q and F : Q → Q are monotone endofunctors such that F = F (the bridge commutes with propagation), then:

```
(lfp(F))  lfp(F)
```

and if additionally F F , then:

```
(lfp(F)) = lfp(F)
```

This is the soundness theorem for cross-domain bridges. It guarantees that fixpoints computed in one domain (e.g., session types) map correctly to fixpoints in another domain (e.g., effect positions).

5.4 4.4 Corollary: Confluence Under Non-Overlapping Resolution

For System III (type checker), the barrier operation (trait resolution commitment) is stratifiable if and only if the instance resolution function is *deterministic* — i.e., for each (trait, type-args) pair, there is at most one valid instance.

Under the non-overlapping instance invariant enforced by Prologos (monomorphic lookup by key, parametric most-specific-wins, same-specificity ties rejected as HKT-7 ambiguity), the resolution function is deterministic, hence stratifiable, hence the stratified fixpoint is confluent.

This result was verified empirically by Track 2 Phase 9 (19 confluence tests, randomized evaluation order, deterministic results).

6 5. Relationship to Approximation Fixpoint Theory

6.1 5.1 The Connection

Approximation Fixpoint Theory (AFT), developed by Denecker, Marek, and Truszczyński (2000, 2004), provides an algebraic framework for fixpoints of non-monotone operators using *approximating operators* on bilattices.

The key construction: given a non-monotone operator $O : L \to L$ on a complete lattice L, define an approximating operator $A : L^2 \to L^2$ on the bilattice L^2 (pairs (x, y) where x y, ordered by the *precision* ordering $_p$), such that A is monotone with respect to $_p$. The fixpoints of A approximate the fixpoints of O.

6.2 5.2 Layered Recovery as a Special Case of AFT

Our stratified recovery systems can be seen as a restricted case of AFT where:

1. The approximating operator A decomposes into a chain of Galois connections (our inter-stratum functors).
2. The bilattice structure is implicit in the stratification: the lower bound of the approximation is the fixpoint of the monotone phase ($\mathrm{lfp}(F_m)$), and the upper bound is determined by the barrier operation.
3. The precision ordering corresponds to the number of strata computed: more strata = more precise approximation.

However, Layered Recovery is *more structured* than general AFT because:

- The monotone and non-monotone phases are cleanly separated (not interleaved).
- The quantale enrichment provides sequential composition, which AFT does not require.
- The Galois connections between strata provide soundness guarantees that general approximating operators do not.

6.3 5.3 What AFT Offers for Future Extensions

AFT provides semantics for operations that Layered Recovery currently does not handle well:

- **Well-founded semantics**: For self-referential negation ($P \leftarrow \neg P$), AFT assigns the well-founded model (unknown). Layered Recovery's stratification rejects this as unstratifiable. AFT provides a graceful degradation.
- **Stable models**: For programs with multiple stable fixpoints (non-deterministic choice), AFT characterizes all stable models. Layered Recovery computes one fixpoint (the least). AFT could extend the ATMS layer to enumerate alternatives.
- **Three-valued fixpoints**: AFT's bilattice naturally supports three-valued logic (true, false, unknown). This could extend Prologos's type checker to track constraints whose satisfiability is not yet determined, rather than failing eagerly.

7 6. Variants and Approaches for Non-Monotonic Domains

7.1 6.1 Classification of Non-Monotone Operations

Not all non-monotone operations are stratifiable. We classify them:

7.1.1 Class A: Stratifiable (barrier-compatible)

The operation $f : L \to L$ is stratifiable if $f \;\; \mathrm{lfp}(F)$ is monotone. This means adding more information to the substrate's fixpoint never reverses the barrier operation's output.

Examples:

- Negation-as-failure (when the program is stratifiable)
- Effect execution (when the effect ordering is derived from a monotone source)
- Trait resolution commitment (under non-overlapping instances)

Approach: Direct application of the Layered Recovery Principle. Insert a barrier between the monotone fixpoint and the non-monotone operation.

7.1.2 Class B: Locally Stratifiable (barrier-compatible with context restriction)

The operation f is not globally stratifiable but becomes stratifiable when restricted to a subset of the lattice. This typically happens when non-monotonicity arises only for certain value combinations.

Examples:

- Overlapping instance resolution (stratifiable for non-overlapping subsets)
- Aggregation with non-monotone aggregates (e.g., average — stratifiable when the set being aggregated is fixed)

Approach: *Dynamic stratification*. Partition the lattice at runtime into regions where f is stratifiable, and process each region at a separate barrier. This is analogous to local stratification in Datalog.

7.1.3 Class C: Anti-Monotone (requires bilattice or ATMS)

The operation f is anti-monotone: more input information decreases the output. Classical negation ($\neg$) is the canonical example.

Examples:

- Classical negation (not NAF — actual logical negation)
- Set complement
- Constraint *retraction* (removing a previously-added constraint)

Approach: Use AFT's bilattice construction. Lift the operation to L^2 where the approximating operator is monotone. Or use the ATMS: treat the anti-monotone operation's inputs as hypotheses, compute fixpoints for each hypothesis set, and select the consistent worldview.

7.1.4 Class D: Chaotic (no fixpoint guarantee)

The operation has no monotonicity or anti-monotonicity structure. Applying it repeatedly may oscillate without converging.

Examples:

- Arbitrary function composition with side effects
- Operations that depend on evaluation order (inherently non-confluent)

Approach: *Bounded iteration with fuel.* Iterate the stratified chain up to a fuel limit (as System III does with fuel=100). If convergence is not reached, report an error. This is not a theoretical guarantee — it is a pragmatic safety valve. Alternatively, impose ordering constraints to make the operation deterministic (converting Class D to Class A at the cost of parallelism).

7.2 6.2 The Adaptation Recipe

For any new non-monotonic domain:

1. **Identify the substrate quantale (L, ,)**: What is the lattice of partial information? What is sequential composition? What is the join (information merge)?

2. **Identify the non-monotone operation f**: What operation on L violates monotonicity? Classify it (A, B, C, or D above).

3. **For Class A**: Define the monotone propagation phase F_m and verify stratifiability. Insert a single barrier.

4. **For Class B**: Identify the stratifiable partitions. Design dynamic stratification that detects partition boundaries at runtime.

5. **For Class C**: Choose between AFT (bilattice approximation) and ATMS (hypothetical reasoning). AFT is simpler for pure logic; ATMS is better when the non-monotone operation interacts with choice.

6. **For Class D**: Apply fuel-bounded iteration. Consider whether ordering constraints can reduce to Class A.

7. **If cross-domain interaction is needed**: Define a Galois connection (,) : L M between the new domain and existing domains. Verify that it is a quantale morphism (preserves). Implement as a bridge propagator.

7.3 6.3 Specific Future Applications in Prologos

Domain	Non-Monotone Operation	Class	A
Refinement types	Predicate weakening under subtyping	B	D
Dependent types	Type-level computation with recursion	D (bounded)	Fu
Module system	Re-export shadowing	A	B
Incremental compilation	Definition invalidation	C	A
Proof search	Backtracking	C	A
Probabilistic types	Bayesian conditioning (non-monotone on probabilities)	B	St

8 7. Implications and Conclusions

8.1 7.1 What the Categorical Structure Tells Us

The three implementations of Layered Recovery are not *analogous* — they are *instances of the same mathematical structure*. The common structure is a stratified endofunctor on a quantale-enriched lattice, with Galois connections between strata and a non-monotone barrier at exactly one point in the chain.

This is not a coincidence. The CALM theorem (Hellerstein 2011) tells us that non-monotone operations require coordination, and that monotone operations do not. The Layered Recovery Principle is the *minimal* coordination strategy: all monotone reasoning happens coordination-free in the propagator network, and the single barrier provides the coordination required by the CALM theorem. The quantale enrichment ensures that sequential structure is preserved across domain bridges.

8.2 7.2 What This Means for Implementation

For implementers, the categorical structure provides:

1. **A checklist**: For each new domain, identify the lattice, the composition, the non-monotone operation, and its class. The categorical structure tells you exactly what infrastructure is needed.

2. **A correctness criterion**: If your Galois connection is a quantale morphism and your barrier operation is stratifiable, confluence is guaranteed. If any of these conditions fails, the structure tells you exactly where the problem is.

3. **A composition principle**: Galois connections compose. Bridge propagators compose. New domains can be added to the propagator network without modifying existing domains — just add a bridge.

8.3 7.3 What This Means for Theorists

The contribution is the identification of quantale enrichment as the key structure that distinguishes Layered Recovery from general stratified fixpoint computation. Lattice-theoretic stratification (Datalog, AFT) handles the fixpoint side. The quantale morphism handles the *sequential composition* side — which is essential for programming language semantics where operations have causal order.

The three systems demonstrate that the same categorical pattern arises independently in logic programming (unification quantale), protocol verification (session quantale), and type inference (substitution quantale). This suggests the pattern is fundamental to any system that combines monotone constraint propagation with non-monotone decision procedures.

8.4 7.4 Open Questions

1. **Is there a 2-categorical structure?** The Galois connections between domains are 1-cells in a 2-category. Do the bridge propagators form 2-cells? If so, what additional coherence conditions arise?

2. **Can the ATMS be characterized as a specific stratification?** Currently, the ATMS is treated as a separate layer. Can it be unified into the stratified endofunctor framework — perhaps as an internal hom in the quantale?

3. **What is the complexity of stratifiability checking?** For System III, we verify stratifiability by enforcing non-overlapping instances (a syntactic condition). Is there a general decision procedure for stratifiability of barrier operations?

4. **Quantaloid enrichment**: When multiple quantales interact via multiple Galois connections, the natural home is a *quantaloid* (a category enriched over quantales). Does the full Prologos propagator network, with all its cross-domain bridges, form a quantaloid-enriched category?

9 8. References

9.1 8.1 Categorical and Algebraic Foundations

- Rosenthal, K.I. (1990). *Quantales and Their Applications*. Longman.

- Stubbe, I. (2013). "An introduction to quantaloid-enriched categories." PDF
- Paseka, J. & Rosický, J. (2000). "Quantales." In *Current Research in Operational Quantum Logic*. ScienceDirect
- Fong, B. & Spivak, D.I. (2019). *Seven Sketches in Compositionality*. §1.3 Galois Connections. LibreTexts

9.2 8.2 Effect Systems and Graded Monads

- Katsumata, S. (2014). "Parametric effect monads and semantics of effect systems." *POPL 2014*. ResearchGate
- Gordon, C.S. (2021). "Lifting Sequential Effects to Indexed Computation Types." *ESOP 2021*.
- Orchard, D. et al. (2020). "Unifying graded and parameterised monads." arXiv
- Uustalu, T. (2018). "Graded monads and quantified computational effects." Slides
- nLab. "Graded monad." nLab
- Barreto, S. & Milius, S. (2025). "A Category-Theoretic Framework for Dependent Effect Systems." arXiv

9.3 8.3 Fixpoint Theory and Stratification

- Cousot, P. & Cousot, R. (1977). "Abstract interpretation: a unified lattice model for static analysis." *POPL 1977*.
- Cousot, P. & Cousot, R. (2014). "A Galois connection calculus for abstract interpretation." *POPL 2014*. PDF
- Denecker, M. et al. (2000, 2004). Approximation Fixpoint Theory. Recent extension
- Heyninck, J. et al. (2025). "A Category-Theoretic Perspective on Approximation Fixpoint Theory." arXiv Cambridge

9.4 8.4 CALM Theorem and Distributed Monotonicity

- Hellerstein, J.M. (2010, 2019). "Keeping CALM: When Distributed Consistency is Easy." *CACM*. arXiv ACM
- Conway, N. et al. (2012). "Logic and Lattices for Distributed Programming." PDF
- Alvaro, P. et al. (2011). "Consistency Analysis in Bloom: a CALM and Collected Approach." *CIDR 2011*. PDF

9.5 8.5 Propagator Networks

- Radul, A. & Sussman, G.J. (2009). "The Art of the Propagator." MIT CSAIL Tech Report.
- Radul, A. (2009). *Propagation Networks: A Flexible and Expressive Substrate for Computation*. PhD thesis, MIT.

9.6 8.6 Session Types

- Caires, L. & Pfenning, F. (2010). "Session Types as Intuitionistic Linear Propositions." *CONCUR 2010*.
- Wadler, P. (2012). "Propositions as Sessions." *ICFP 2012*.
- Atkey, R. (2009). "Parameterised Notions of Computation." *JFP 19(3-4)*.

9.7 8.7 Datalog and Non-Monotonic Reasoning

- Madsen, M. et al. (2024). "From Datalog to Flix: A Declarative Language for Fixed Points on Lattices." Berkeley
- Charalambidis, A. et al. (2025). "The Power of Negation in Higher-Order Datalog." *TPLP*. Cambridge
- Kuper, L. & Newton, R.R. (2023). "Breaking the Negative Cycle." *ECOOP 2023*. PDF

9.8 8.8 Prologos Internal Documents

- `EFFECTFUL_COMPUTATION_ON_PROPAGATORS.org` — Layered Recovery Principle (canonical)
- `2026-03-06_SESSION_TYPES_AS_EFFECT_ORDERING.org` — System II design
- `2026-02-24_LOGIC_ENGINE_DESIGN.org` — System I design
- `2026-03-13_TRACK2_REACTIVE_RESOLUTION_DESIGN.md` — System III design
- `2026-02-27_1026_GALOIS_CONNECTIONS_ABSTRACT_INTERPRETATION.md` — Galois connections research
- `2026-03-11_PROPAGATOR_FIRST_PIPELINE_AUDIT.md` — Full system audit
- `DESIGN_PRINCIPLES.org` — Data Orientation principle